

Part A: Conceptual Foundation (Theory) — Answers

Regularization, Cross-Validation & Tree-Based Models

Q1. What is Regularization in Machine Learning? Why is it needed?

Regularization is a technique that adds a penalty term to a model's loss function based on the size of its coefficients. This discourages the model from fitting the training data too closely and keeps the learned weights small and simple.

General form:

$$\text{Loss}_{\text{regularized}} = \text{Loss}_{\text{original}} \text{ (e.g. MSE)} + \lambda \times \text{Penalty}(\text{coefficients})$$

Why it is needed:

- Prevents **overfitting** — without a penalty, a model can fit noise in the training data, giving low training error but poor performance on new data.
- Controls model **complexity** by shrinking large coefficients, producing a smoother, more generalizable decision boundary.
- Improves **stability** when features are correlated (multicollinearity), where unregularized coefficients can become very large or unstable.

Why Regularization is Needed: Bias-Variance Trade-off

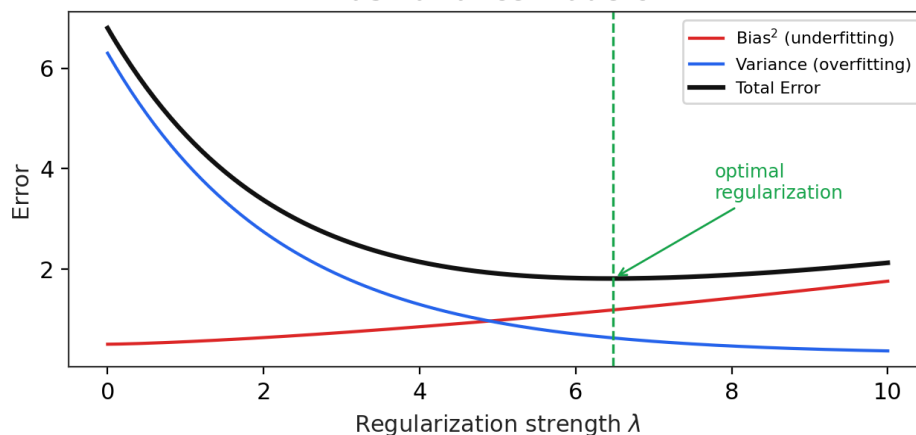


Fig 1: As λ increases, variance (overfitting) falls but bias (underfitting) rises — regularization finds the sweet spot that minimizes total error.

Q2. Difference between Ridge Regression (L2) and Lasso Regression (L1)

Ridge Regression (L2 penalty):

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \lambda \sum w_j^2$$

Lasso Regression (L1 penalty):

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \lambda \sum |w_j|$$

Aspect	Ridge (L2)	Lasso (L1)
Penalty term	Sum of squared coefficients	Sum of absolute coefficients

Effect on coefficients	Shrinks toward zero, rarely = 0	Can shrink exactly to zero
Feature selection	No — keeps all features	Yes — performs automatic feature selection
Best used when	Many small/medium-sized relevant features	Few features truly matter (sparse solution needed)
Handles multicollinearity	Yes, very well	Less stable, picks one of correlated features

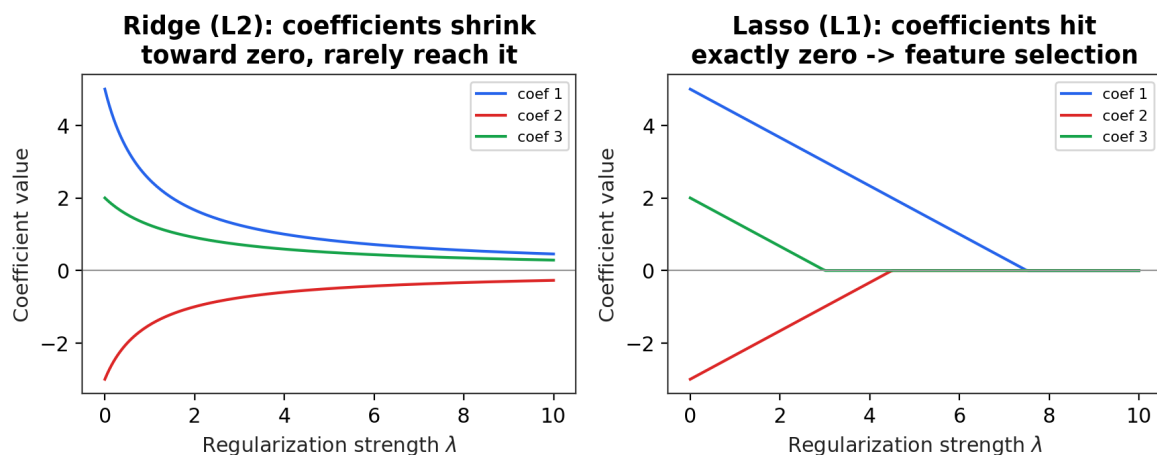


Fig 2: As λ grows, Ridge coefficients shrink smoothly toward zero, while Lasso coefficients hit exactly zero one by one — this is why Lasso performs feature selection.

Q3. What is Cross-Validation and why is it important?

Cross-validation is a resampling technique used to evaluate how well a model generalizes to unseen data. The dataset is repeatedly split into training and validation subsets, the model is trained and tested on each split, and the results are averaged to get a reliable performance estimate.

Why it is important:

- Gives a more **reliable estimate** of model performance than a single train/test split.
- Helps detect **overfitting** — a model that performs well on training data but poorly across validation folds is likely overfit.
- Makes efficient use of **limited data**, since every observation gets used for both training and validation.
- Useful for **hyperparameter tuning** (e.g. choosing λ in Ridge/Lasso) by comparing average validation scores across folds.

Q4. Cross-Validation Techniques

(a) K-Fold Cross-Validation

The dataset is split into K equal-sized folds. The model is trained on K–1 folds and validated on the remaining fold; this is repeated K times so every fold is used once for validation. The final score is the average across all K runs.

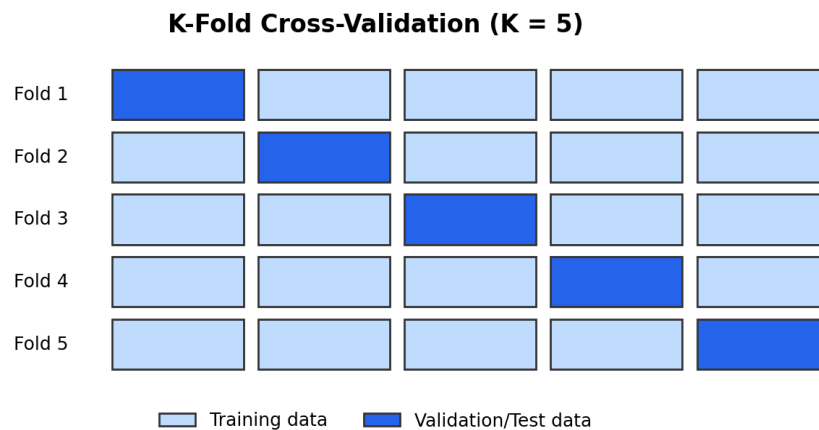


Fig 3: In 5-fold CV, each row is one run — the dark block is the validation fold.

(b) Stratified K-Fold Cross-Validation

Same as K-Fold, but each fold is created so that it preserves the same class proportions as the full dataset. This is essential for classification problems with imbalanced classes, so that no fold ends up with too few (or zero) samples of a minority class.

(c) Leave-One-Out Cross-Validation (LOOCV)

An extreme case of K-Fold where $K = n$ (number of data points). Each iteration trains on all but one sample and validates on that single left-out sample. It is very thorough but computationally expensive for large datasets, since the model must be trained n times.

(d) Time Series Split

Used for time-ordered/sequential data, where random shuffling would leak future information into the past. The training set only ever contains data that comes before the validation set in time, and the training window grows (or slides) forward with each split.

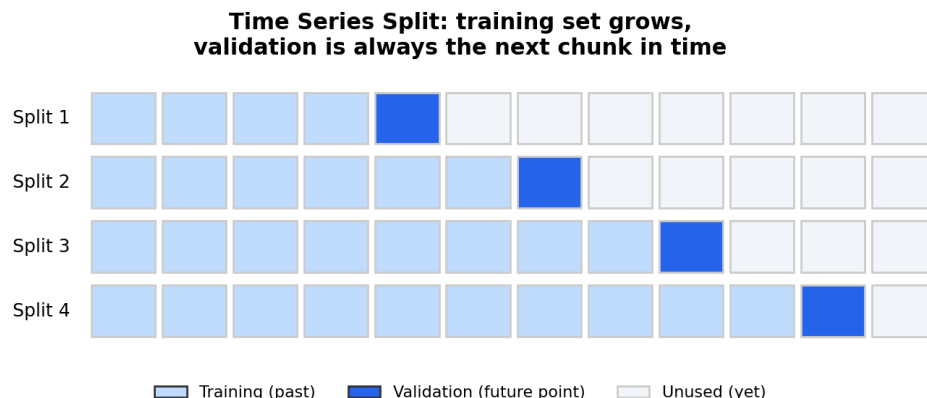


Fig 4: Training data always precedes the validation point in time — no shuffling, no future leakage.

Q5. Why are tree-based models less sensitive to feature scaling?

Tree-based models (Decision Trees, Random Forest, Gradient Boosting/XGBoost, etc.) choose splits by comparing feature values against a threshold, for example: **Feature X > c**. This decision depends only on the **rank order** of the values, not their absolute magnitude or units.

- Scaling a feature (e.g. standardizing or normalizing it) is a **monotonic transformation** — it preserves the order of values, so the same split point still separates the data identically.

- Distance-based models (KNN, SVM, linear/logistic regression with regularization, K-Means) compute distances or weighted sums across features, so features on larger scales dominate — trees never compute such distances, so this problem doesn't arise.
- As a result, trees give identical predictions whether the input is in raw units, standardized units, or normalized to $[0,1]$.

Tree splits depend only on rank/order, not magnitude

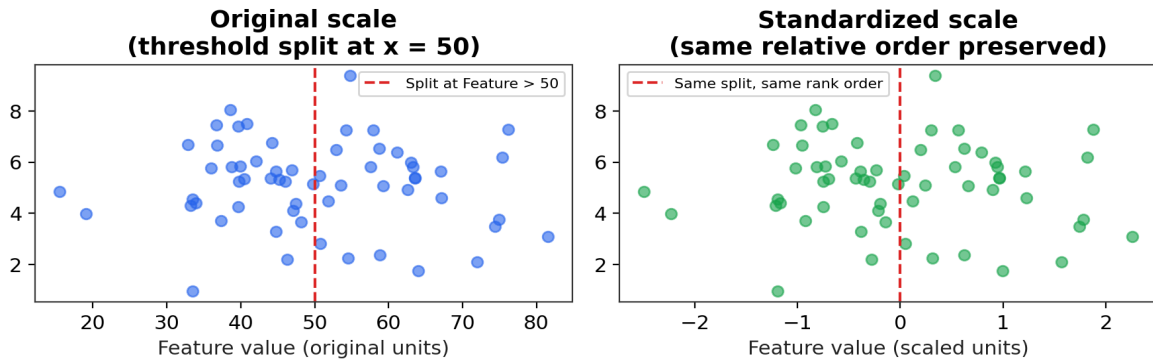


Fig 5: The split threshold moves with the data ($x = 50$ vs $x = 0$ standardized), but it still separates exactly the same points on either side — the split decision is unchanged.